



aqua-ai-monitor

Plateforme Intelligente de Suivi en Aquaculture

Rapport de Projet — Systèmes et Projets Industriels

Établissement	CESI Nanterre
Programme	FISA — Ingénierie des Systèmes Embarqués
Promotion	2024-2027
Encadrant	Kamel AIZI

Équipe projet

DOGOTARU Arthur	Chef de projet — GUI, IA
LEGALL Maxence	Capteurs, traitement de données
HAMMOUDA Nouhaila	PCB, schématique électronique

19 mai 2026

Résumé

Le secteur de l'aquaculture est confronté à des défis croissants en matière de surveillance continue des conditions d'élevage. Les variations de paramètres physicochimiques tels que le pH ou la turbidité de l'eau peuvent provoquer des pertes importantes en quelques heures, sans que l'opérateur soit nécessairement présent pour les détecter.

Le projet **aqua-ai-monitor** propose une réponse à cette problématique en développant une plateforme embarquée autonome, combinant acquisition de données IoT, traitement temps réel et intelligence artificielle. Le système repose sur deux microcontrôleurs ESP32 — l'un dédié à l'acquisition sensorielle (température, pH, turbidité, humidité), l'autre à la capture vidéo par caméra OV2640 — qui transmettent leurs données par Wi-Fi à un serveur central.

Ce serveur, développé en Node.js, agrège les flux de données, les enregistre dans un fichier CSV, et les transmet à un microservice Python hébergeant un modèle *Isolation Forest* entraînable à la volée. La détection d'anomalies repose sur une double logique : seuils métier configurable et score d'anomalie statistique. Les résultats sont présentés via un tableau de bord web en temps réel, incluant graphiques d'évolution temporelle, alertes colorées et module d'entraînement par upload CSV.

Mots-clés : ESP32, IoT, aquaculture, détection d'anomalies, Isolation Forest, Node.js, Flask, Chart.js, KiCad, PlatformIO.

Table des matières

Résumé	1
Liste des figures	5
Liste des tableaux	6
1 Introduction et contexte	7
1.1 Présentation du projet	7
1.2 Problématique industrielle	7
1.3 Objectifs du projet	8
1.4 Structure du rapport	8
2 Architecture générale du système	9
2.1 Vue d'ensemble	9
2.2 Flux de données	10
2.3 Choix technologiques	10
2.4 Protocole de communication ESP32 → Serveur	11
3 Matériel électronique et conception PCB	12
3.1 Composants utilisés	12
3.2 Capteur de pH — PH-4502C	12
3.2.1 Principe de fonctionnement	12
3.2.2 Implémentation sur ESP32	13
3.2.3 Filtrage numérique	13
3.3 Capteur de turbidité — Keyestudio V1.0	13
3.4 Capteur DS18B20	14
3.5 Conception PCB — KiCad	14
3.5.1 Objectifs de la carte	14
3.5.2 Caractéristiques techniques	15
3.5.3 Fichiers de fabrication	15
4 Firmware ESP32	16
4.1 Organisation du projet PlatformIO	16
4.2 Boucle principale non-bloquante	16
4.3 Configuration centralisée	17
4.4 Envoi HTTP	17
4.5 Firmware ESP32-CAM	18

4.5.1	Initialisation de la caméra OV2640	18
4.5.2	Streaming MJPEG par chunks HTTP	18
5	Serveur Node.js	19
5.1	Architecture du serveur	19
5.2	Enregistrement CSV automatique	19
5.3	Intégration du microservice IA	20
5.4	Proxy MJPEG	21
6	Module d'intelligence artificielle	22
6.1	Problématique de la détection d'anomalies	22
6.2	Choix de l'algorithme : Isolation Forest	22
6.2.1	Principe	22
6.2.2	Comparaison avec les alternatives	23
6.3	Architecture du microservice Flask	23
6.4	Pipeline d'entraînement	23
6.5	Pipeline d'inférence	24
6.6	Persistance du modèle	25
6.7	Workflow d'utilisation	25
7	Tableau de bord web	26
7.1	Architecture frontend	26
7.2	Organisation en onglets	26
7.2.1	Onglet Dashboard	26
7.2.2	Onglet Graphiques	26
7.3	Système de polling	26
7.4	Gestion des alertes	27
7.5	Interface d'entraînement IA	27
8	Résultats et validation	28
8.1	Tests du firmware	28
8.1.1	Lecture des capteurs	28
8.2	Tests du modèle IA	28
8.3	Performance du système	29
9	Conclusion et perspectives	30
9.1	Bilan du projet	30
9.2	Compétences développées	30
9.3	Perspectives d'amélioration	31
9.3.1	Court terme	31
9.3.2	Moyen terme	31

9.3.3 Long terme	31
Références bibliographiques	32
A Annexe A — Schéma de câblage ESP32	34
B Annexe B — Guide de démarrage rapide	35
B.1 Prérequis	35
B.2 Installation	35
B.3 Flash du firmware ESP32	35
B.4 Premier entraînement du modèle	36
C Annexe C — Structure du dépôt GitHub	37

Table des figures

2.1 Schéma de l'architecture du banc de test	9
--	---

Liste des tableaux

2.1	Tableau des choix technologiques	10
3.1	Inventaire des composants matériels	12
3.2	Caractéristiques de la carte PCB carte_aqua	15
4.1	Paramètres de configuration de la caméra OV2640	18
5.1	Endpoints du serveur Node.js	19
6.1	Comparaison des algorithmes de détection d'anomalies	23
8.1	Résultats de validation des capteurs	28
8.2	Résultats de prédiction du modèle IA sur données test	29
8.3	Métriques de performance du système complet	29
9.1	Bilan des objectifs	30
A.1	Câblage complet ESP32 – capteurs	34

Chapitre 1

Introduction et contexte

1.1 Présentation du projet

Le projet **aqua-ai-monitor** est un système de surveillance automatisée de bassins aquacoles développé dans le cadre du projet de Systèmes et Projets Industriels (SPI) de deuxième année FISA au CESI Nanterre. Il vise à concevoir une plateforme complète, de la couche matérielle embarquée jusqu'à l'interface utilisateur et l'intelligence artificielle, répondant à une problématique industrielle réelle.

L'équipe est composée de trois étudiants aux compétences complémentaires :

- **Arthur Dogotaru** — Chef de projet, responsable de l'interface graphique (GUI), du microservice d'intelligence artificielle et de l'intégration logicielle globale ;
- **Legall Maxence** — Responsable de l'intégration des capteurs et du traitement des données embarquées ;
- **Hammouda Nouhaila** — Responsable de la conception PCB et des schémas électroniques sous KiCad.

1.2 Problématique industrielle

L'aquaculture représente une part croissante de la production mondiale de protéines animales. Selon la FAO, elle couvre aujourd'hui plus de 50 % de la consommation mondiale de poissons. La gestion des bassins d'élevage constitue cependant un défi opérationnel majeur : les organismes aquatiques sont extrêmement sensibles aux variations de leur environnement.

Risques liés à la surveillance manuelle

Un écart de pH de 0,5 unité peut entraîner un stress physiologique significatif chez les poissons. Une turbidité excessive indique une prolifération bactérienne ou une contamination. Sans surveillance continue, ces anomalies peuvent provoquer des pertes totales d'un élevage en moins de 24 heures.

La surveillance manuelle présente trois limitations fondamentales :

1. **Discontinuité** : les relevés manuels sont typiquement effectués une à deux fois par jour, laissant de larges fenêtres de non-détection ;

2. **Réactivité** : une intervention humaine est nécessaire pour interpréter les mesures et déclencher une alerte, introduisant un délai incompressible ;
3. **Coût** : la mobilisation permanente de personnel qualifié représente une charge opérationnelle élevée pour les petites et moyennes exploitations aquacoles.

1.3 Objectifs du projet

Le projet aqua-ai-monitor vise à répondre à ces trois limitations par un système autonome capable de :

- Collecter en continu les paramètres physicochimiques (pH, turbidité, températures, humidité) via des capteurs connectés à un microcontrôleur ESP32 ;
- Transmettre ces données par Wi-Fi à un serveur central via le protocole HTTP/JSON ;
- Enregistrer les données dans un fichier CSV pour constitution d'un historique ;
- Analyser les données à l'aide d'un modèle d'apprentissage automatique non supervisé (Isolation Forest) pour détecter les anomalies sans nécessiter de labellisation préalable ;
- Présenter les résultats en temps réel sur un tableau de bord web accessible depuis n'importe quel navigateur sur le réseau local ;
- Capturer un flux vidéo sous-marin pour l'observation comportementale des poissons.

1.4 Structure du rapport

Ce rapport est organisé en sept chapitres. Le **chapitre 2** présente l'architecture générale du système et les choix technologiques. Le **chapitre 3** décrit le matériel électronique et la carte PCB. Le **chapitre 4** détaille les firmwares ESP32. Le **chapitre 5** présente le serveur Node.js et l'infrastructure logicielle. Le **chapitre 6** développe le module d'intelligence artificielle. Le **chapitre 7** présente le tableau de bord web. Le **chapitre 8** conclut et ouvre sur les perspectives.

Chapitre 2

Architecture générale du système

2.1 Vue d'ensemble

Le système aqua-ai-monitor est structuré en trois couches fonctionnelles distinctes, conformément au modèle IoT classique :

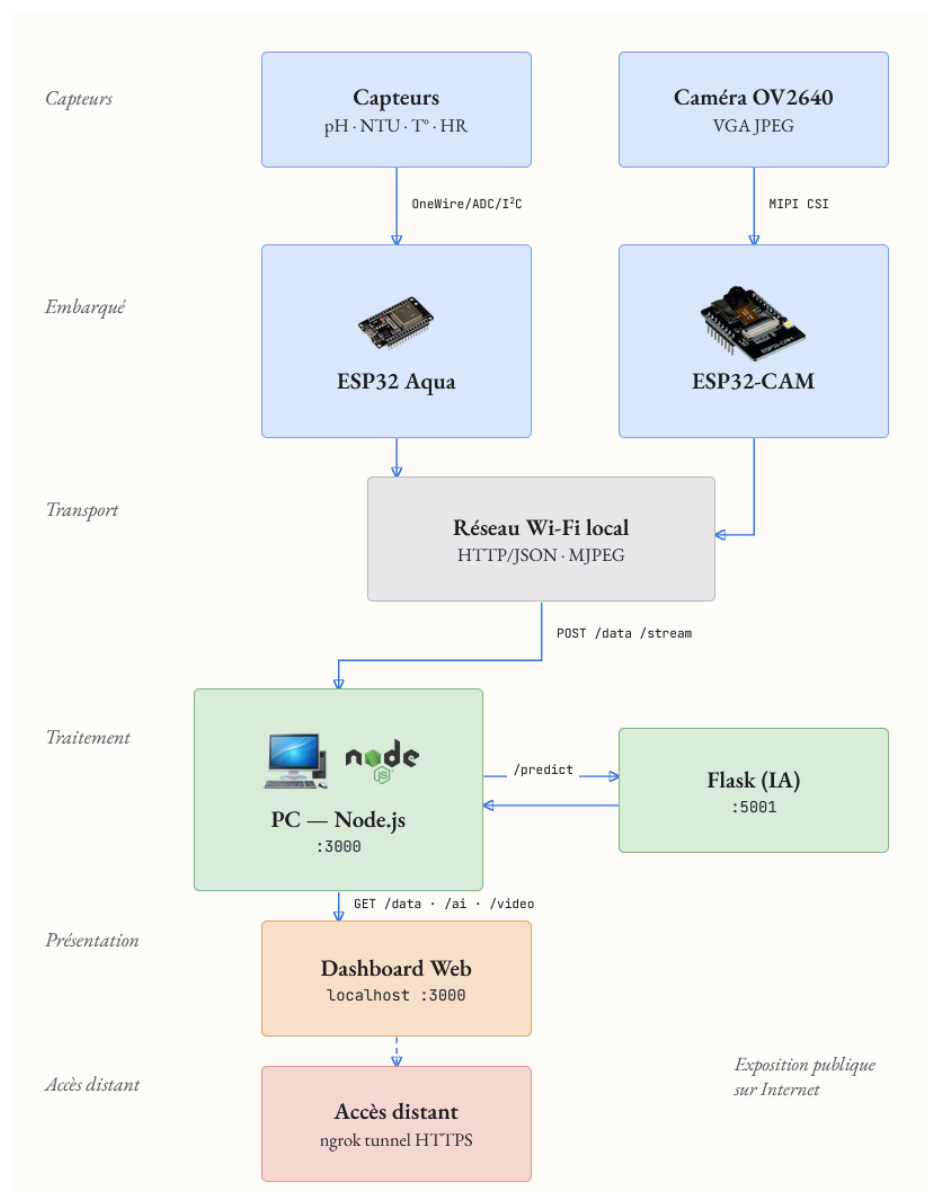


FIGURE 2.1. Schéma de l'architecture du banc de test

2.2 Flux de données

Le flux de données suit le chemin suivant :

```
Capteurs → ESP32 Aqua → HTTP POST /data → Node.js →
POST /predict (Flask) → Dashboard
```

En parallèle, le flux vidéo emprunte :

```
OV2640 → ESP32-CAM → HTTP POST /stream (chunks) → Node.js →
GET /video (MJPEG) → Dashboard
```

2.3 Choix technologiques

TABLE 2.1. Tableau des choix technologiques

Composant	Technologie	Justification
Microcontrôleur	ESP32-WROOM-32	Wi-Fi intégré, ADC 12 bits, bibliothèques Arduino riches, faible coût
Firmware	C++/PlatformIO	Abstraction Arduino, gestion de projets multi-fichiers, OTA possible
Communication	HTTP/JSON	Simplicité d'intégration, pas de broker requis, compatible LAN
Serveur	Node.js/Express	Asynchrone natif, adapté aux flux temps réel, npm riche
IA	Python/Flask	Écosystème scikit-learn mature, Flask léger pour microservice
Modèle IA	Isolation Forest	Non supervisé, efficace sur peu de variables, interprétable
Dashboard	HTML/JS vanilla + Chart.js	Pas de dépendance framework, déploiement immédiat
PCB	KiCad 7	Open-source, export Gerber standard, bibliothèques communautaires

2.4 Protocole de communication ESP32 → Serveur

L'ESP32 aqua envoie une requête HTTP POST toutes les 2 secondes vers l'endpoint `/data` du serveur. Le corps de la requête est un objet JSON :

```
1 {  
2   "temperature": 20.4,    // Temperature air (deg C) - DHT22  
3   "humidity":    60.2,    // Humidite relative (%) - DHT22  
4   "ph":          7.23,    // pH de l'eau - capteur PH-4502C  
5   "ntu":         542.0,    // Turbidite (NTU) - Keyestudio V1.0  
6   "waterTemp":   17.87    // Temperature eau (deg C) - DS18B20  
7 }
```

Listing 2.1. Structure JSON envoyée par l'ESP32

L'ESP32-CAM envoie quant à lui un flux HTTP chunked en `Transfer-Encoding: chunked`, dans lequel chaque chunk correspond à une frame JPEG capturée par l'OV2640.

Chapitre 3

Matériel électronique et conception PCB

3.1 Composants utilisés

TABLE 3.1. Inventaire des composants matériels

Composant	Référence	Interface	Paramètre mesuré
Microcontrôleur principal	ESP32-WROOM-32	Wi-Fi, GPIO	Coordination, transmission
Microcontrôleur caméra	ESP32-CAM	Wi-Fi, MIPI	Vidéo sous-marine
Capteur température eau	DS18B20	OneWire (GPIO 4)	Température eau (°C)
Capteur température/humidité	DHT22	Single-wire (GPIO 15)	Temp. air (°C), humidité (%)
Capteur pH	PH-4502C	ADC (GPIO 34)	pH (0–14)
Capteur turbidité	Keystudio V1.0	ADC (GPIO 35)	Turbidité (NTU)
Caméra	OV2640	MIPI CSI	Image JPEG VGA
Écran OLED	SSD1306 128×64	I ² C (GPIO 21/22)	Affichage local

3.2 Capteur de pH — PH-4502C

3.2.1 Principe de fonctionnement

Le capteur PH-4502C est basé sur une électrode de verre combinée. La différence de potentiel développée par l'électrode suit l'équation de Nernst :

$$E = E_0 + \frac{RT}{nF} \ln[\text{H}^+] = E_0 - \frac{RT}{F} \cdot \text{pH} \quad (3.1)$$

où $R = 8,314 \text{ J}/(\text{mol}\cdot\text{K})$, T la température en Kelvin, $F = 96485 \text{ C}/\text{mol}$ et $n = 1$.

3.2.2 Implémentation sur ESP32

L'ESP32 lit la tension en sortie du module via son ADC 12 bits (résolution 4095 niveaux, plage 0–3,3 V après atténuation `ADC_11db`). La conversion tension $\rightarrow$ pH est réalisée par une droite affine étalonnée sur deux points de référence :

$$\text{pH} = 9,18 + \frac{0,781 - V}{0,122} + \text{offset} \quad (3.2)$$

Les points de calibration utilisés sont :

- $V = 0,781 \text{ V} \rightarrow \text{pH } 9,18$ (solution tampon basique);
- $V = 1,388 \text{ V} \rightarrow \text{pH } 4,00$ (solution tampon acide).

3.2.3 Filtrage numérique

Pour réduire le bruit de conversion, chaque mesure de pH est obtenue par moyenne sur $N = 10$ échantillons espacés de 10 ms :

```

1  float PHSensor::read() {
2      long sum = 0;
3      for (int i = 0; i < PH_SAMPLES; i++) {
4          sum += analogRead(PH_PIN);
5          delay(10);
6      }
7      float raw      = sum / (float)PH_SAMPLES;
8      float voltage = (raw / 4095.0f) * PH_VREF;
9      return _voltageToPhValue(voltage);
10 }
```

Listing 3.1. Filtrage par moyennage – `ph_sensor.cpp`

3.3 Capteur de turbidité — Keyestudio V1.0

Le capteur de turbidité fonctionne par mesure de la diffusion lumineuse (principe néphélométrique). La tension de sortie est inversement proportionnelle à la turbidité : une eau claire produit une tension élevée ($\approx 1,533 \text{ V}$), une eau très trouble une tension proche de 0 V.

$$\text{NTU} = (1,533 - V) \times \frac{3000}{1,533} \quad (3.3)$$

Limite du modèle linéaire

La relation linéaire proposée est une approximation. Pour des mesures précises au-delà de 500 NTU, une calibration multi-points avec des solutions étalons

certifiées (normes ISO 7027) est recommandée.

3.4 Capteur DS18B20

Le DS18B20 est un capteur de température numérique à protocole OneWire, étanche, utilisé pour la mesure de température de l'eau. Sa résolution est configurable de 9 à 12 bits ($\pm 0,5^{\circ}\text{C}$ à $\pm 0,0625^{\circ}\text{C}$). La bibliothèque `DallasTemperature` gère l'intégralité du protocole OneWire, permettant un code applicatif simple :

```

1 float DS18B20Sensor::read() {
2   _sensors.requestTemperatures();
3   float temp = _sensors.getTempCByIndex(0);
4   if (temp == DEVICE_DISCONNECTED_C) return -127.0f;
5   return temp;
6 }
```

Listing 3.2. Lecture DS18B20 – ds18b20_sensor.cpp

La valeur sentinelle `-127.0f` permet de détecter une déconnexion physique du capteur et de l'indiquer dans l'interface.

3.5 Conception PCB — KiCad

3.5.1 Objectifs de la carte

La carte PCB *carte_aqua*, conçue par Hammouda Nouhaila sous KiCad 7, a pour objectif de regrouper sur un circuit imprimé les connecteurs nécessaires à l'ensemble des capteurs, l'alimentation et l'ESP32-WROOM-32 DevKit V1. Elle supprime le câblage sur breadboard pour un déploiement fiable en environnement humide.

3.5.2 Caractéristiques techniques

TABLE 3.2. Caractéristiques de la carte PCB carte_aqua

Caractéristique	Valeur
Outil de conception	KiCad 7
Nombre de couches	2 (F_Cu, B_Cu)
Fichiers de sortie	Gerber (BRD, F/B_Cu, F/B_Mask, F/B_Silk, Edge_Cuts)
Alimentation	5 V / 2 A (USB ou bloc secteur)
Microcontrôleur intégré	ESP32-WROOM-32 DevKit V1 (empreinte 3D incluse)
Connecteurs capteurs	JST-XH 2.54 mm (DS18B20, DHT22, pH, turbidité, OLED)

3.5.3 Fichiers de fabrication

Les fichiers Gerber exportés permettent une fabrication directe chez tout fabricant PCB standard (JLCPCB, PCBWay, etc.). Les couches exportées comprennent : F_Cu, B_Cu, F_Mask, B_Mask, F_Silkscreen, B_Silkscreen et Edge_Cuts.

Chapitre 4

Firmware ESP32

4.1 Organisation du projet PlatformIO

Le firmware est développé avec PlatformIO sous VS Code, ciblant la plateforme `espressif32` avec le framework Arduino. L'organisation suit le modèle orienté objet, avec une classe dédiée par capteur ou sous-système :

```
1 esp32_aqua/  
2   src/  
3     main.cpp          -- Boucle principale  
4     config.h          -- Constantes de configuration  
5     dht_sensor.cpp/h  -- Classe DHTSensor  
6     ds18b20_sensor.cpp/h -- Classe DS18B20Sensor  
7     ph_sensor.cpp/h   -- Classe PHSensor  
8     turbidity_sensor.cpp/h -- Classe TurbiditySensor  
9     screen_oled.cpp/h -- Classe ScreenOLED  
10    http_sender.cpp/h -- Classe HttpSender  
11    platformio.ini     -- Configuration build
```

Listing 4.1. Structure du firmware esp32_aqua

4.2 Boucle principale non-bloquante

La boucle `loop()` est conçue sans aucun appel bloquant global. Chaque classe expose une méthode `isReady()` qui retourne `true` lorsque l'intervalle de lecture configuré est écoulé depuis la dernière mesure. Cette approche évite l'utilisation de tâches FreeRTOS tout en maintenant une réactivité suffisante.

```
1 void loop() {  
2   if (dhtSensor.isReady())      lastData      = dhtSensor.read();  
3   if (phSensor.isReady())       lastPh        = phSensor.read();  
4   if (turbSensor.isReady())     lastNtu       = turbSensor.read();  
5   if (waterTempSensor.isReady()) lastWaterTemp = waterTempSensor.read  
6                                   ();  
7   if (lastData.valid) {  
8     screen.showData(lastData, lastPh, lastNtu, lastWaterTemp);  
9     if (sender.isReady())
```

```

10     sender.send(lastData, lastPh, lastNtu, lastWaterTemp);
11 } else {
12     screen.showError("DHT22 KO");
13 }
14 }

```

Listing 4.2. Boucle principale non-bloquante – main.cpp

4.3 Configuration centralisée

Toutes les constantes de configuration (SSID, mot de passe Wi-Fi, adresse serveur, broches GPIO, intervalles de lecture) sont regroupées dans `config.h` :

```

1 #define WIFI_SSID      "MonReseau"
2 #define WIFI_PASSWORD  "MotDePasse"
3 #define SERVER_URL     "http://192.168.1.73:3000/data"
4 #define DHTPIN         15
5 #define PH_PIN         34
6 #define TURB_PIN       35
7 #define DS18B20_PIN    4
8 #define PH_READ_INTERVAL_MS 2000
9 #define TURB_READ_INTERVAL_MS 2000

```

Listing 4.3. Extrait de config.h

Sécurité des credentials

Les identifiants Wi-Fi sont actuellement stockés en clair dans `config.h`. Pour un déploiement en production, il est recommandé d'utiliser le mécanisme de provisionnement Wi-Fi d'Espressif (SmartConfig ou BLE provisioning) ou de stocker les credentials dans la partition NVS de l'ESP32.

4.4 Envoi HTTP

La classe `HttpSender` utilise la bibliothèque `HTTPClient` d'Arduino pour ESP32. Le corps de la requête est construit manuellement en JSON pour éviter la dépendance à une bibliothèque JSON volumineuse :

```

1 void HttpSender::send(const SensorData& data,
2                       float ph, float ntu, float waterTemp) {
3     HTTPClient http;
4     http.begin(SERVER_URL);
5     http.addHeader("Content-Type", "application/json");
6
7     String json = "{";

```

```

8   json += "\"temperature\": " + String(data.temperature, 1) + ",";
9   json += "\"humidity\": " + String(data.humidity, 1) + ",";
10  json += "\"ph\": " + String(ph, 2) + ",";
11  json += "\"ntu\": " + String(ntu, 1) + ",";
12  json += "\"waterTemp\": " + String(waterTemp, 2);
13  json += "}";
14
15  int code = http.POST(json);
16  http.end();
17 }

```

Listing 4.4. Construction et envoi du JSON – http_sender.cpp

4.5 Firmware ESP32-CAM

4.5.1 Initialisation de la caméra OV2640

Le firmware de l'ESP32-CAM initialise la caméra avec la configuration suivante :

TABLE 4.1. Paramètres de configuration de la caméra OV2640

Paramètre	Valeur	Commentaire
Format pixel	JPEG	Compression embarquée
Résolution	VGA (640×480)	Compromis qualité/débit
Qualité JPEG	6	0 (max) – 63 (min)
Nombre de buffers	2	Double buffering anti-tearing
Fréquence XCLK	20 MHz	Fréquence standard OV2640

4.5.2 Streaming MJPEG par chunks HTTP

L'ESP32-CAM maintient une connexion TCP persistante vers le serveur Node.js. Chaque frame JPEG est envoyée comme un chunk HTTP :

```

1  bool Streamer::send(camera_fb_t* fb) {
2      if (!_connect()) return false;
3      _client.printf("%x\r\n", fb->len); // Taille chunk en hexa
4      _client.write(fb->buf, fb->len);   // Donnees JPEG
5      _client.print("\r\n");
6      return true;
7  }

```

Listing 4.5. Envoi d'une frame en chunked – streamer.cpp

Chapitre 5

Serveur Node.js

5.1 Architecture du serveur

Le serveur Node.js joue un rôle central d'agrégateur et de proxy. Il est développé avec le framework **Express.js** et expose plusieurs endpoints HTTP sur le port 3000 :

TABLE 5.1. Endpoints du serveur Node.js

Méthode	Route	Description
POST	/data	Réception des données JSON de l'ESP32, appel IA, écriture CSV
GET	/data	Lecture des dernières données pour le dashboard
GET	/ai	Lecture du dernier résultat d'analyse IA
GET	/ai/status	Statut du modèle (entraîné/non entraîné, métriques)
POST	/train	Upload CSV multipart → transfert au microservice Flask
GET	/csv	Téléchargement du fichier de données enregistré
POST	/stream	Réception du flux vidéo chunked de l'ESP32-CAM
GET	/video	Diffusion MJPEG aux clients (Server-Sent Events)
GET	/	Service du dashboard HTML statique

5.2 Enregistrement CSV automatique

À chaque réception de données de l'ESP32, le serveur ajoute une ligne au fichier `data_aqua.csv` :

```
1 const CSV_PATH    = path.join(__dirname, 'data_aqua.csv');
2 const CSV_HEADER = 'timestamp,ph,ntu,temperature,humidity,waterTemp\n'
;
```

```

3
4 if (!fs.existsSync(CSV_PATH))
5   fs.writeFileSync(CSV_PATH, CSV_HEADER);
6
7 function appendCSV(data) {
8   if (data.ph == null || data.ntu == null) return;
9   const line = `${data.updatedAt},${data.ph},${data.ntu},` +
10     `${data.temperature ?? ''},${data.humidity ?? ''},` +
11     `${data.waterTemp ?? ''}\n`;
12   fs.appendFile(CSV_PATH, line, err => {
13     if (err) console.error('[CSV] Erreur:', err.message);
14   });
15 }

```

Listing 5.1. Écriture CSV en mode append – server.js

Ce fichier constitue la base d'entraînement du modèle IA. Il peut être téléchargé via l'endpoint `GET /csv` puis ré-uploadé pour entraîner le modèle via l'interface web.

5.3 Intégration du microservice IA

À chaque réception de données, le serveur effectue une requête asynchrone vers le microservice Python :

```

1 app.post('/data', async (req, res) => {
2   lastSensorData = { ...req.body, updatedAt: new Date().toISOString()
3     };
4   appendCSV(lastSensorData);
5   res.sendStatus(200);
6
7   if (lastSensorData.ph !== null && lastSensorData.ntu !== null) {
8     try {
9       const aiRes = await fetch(`${AI_URL}/predict`, {
10         method: 'POST',
11         headers: { 'Content-Type': 'application/json' },
12         body: JSON.stringify({ ph: lastSensorData.ph,
13           ntu: lastSensorData.ntu })
14       });
15       lastAiResult = { ...await aiRes.json(),
16         updatedAt: new Date().toISOString() };
17     } catch (e) {
18       console.warn('[AI] Service injoignable : ${e.message}');
19     }
20   });

```

Listing 5.2. Appel asynchrone au microservice IA

Découplage réponse HTTP et appel IA

La réponse 200 est envoyée à l'ESP32 avant l'appel IA, afin de ne pas bloquer le prochain cycle d'envoi du firmware. L'appel au microservice Python est entièrement asynchrone.

5.4 Proxy MJPEG

Le proxy vidéo MJPEG est l'un des éléments les plus délicats du serveur. Il reçoit le flux binaire brut de l'ESP32-CAM en `POST /stream`, parse les frames JPEG en détectant les marqueurs SOI (`0xFF 0xD8`) et EOI (`0xFF 0xD9`), puis les redistribue à tous les clients connectés sur `GET /video` en `multipart/x-mixed-replace` :

```
1 req.on('data', chunk => {
2   buffer = Buffer.concat([buffer, chunk]);
3   let start = -1;
4   for (let i = 0; i < buffer.length - 1; i++) {
5     if (buffer[i] === 0xFF && buffer[i+1] === 0xD8) start = i;
6     if (start !== -1 && buffer[i] === 0xFF && buffer[i+1] === 0xD9) {
7       const frame = buffer.slice(start, i + 2);
8       buffer = buffer.slice(i + 2);
9       clients.forEach(client => {
10        client.write('--frame\r\nContent-Type: image/jpeg\r\n\r\n');
11        client.write(frame);
12        client.write('\r\n');
13      });
14      start = -1; i = 0;
15    }
16  }
17 });
```

Listing 5.3. Extraction des frames JPEG

Chapitre 6

Module d'intelligence artificielle

6.1 Problématique de la détection d'anomalies

La détection d'anomalies dans les données de capteurs aquacoles présente plusieurs contraintes spécifiques :

- **Absence de labels** : il n'existe pas, en début de projet, de jeu de données annoté distinguant les situations normales des anomalies. Une approche supervisée est donc impossible sans un effort de labellisation coûteux.
- **Faible dimensionnalité** : seules deux variables sont utilisées pour la détection (pH et NTU), ce qui favorise les méthodes légères.
- **Entraînement à la volée** : le système doit pouvoir être entraîné sur les données réelles collectées dans le bassin cible, sans nécessiter d'infrastructure cloud.
- **Interprétabilité** : les alertes doivent être accompagnées d'une raison compréhensible par l'opérateur.

6.2 Choix de l'algorithme : Isolation Forest

6.2.1 Principe

L'Isolation Forest **liu2008isolation** est un algorithme de détection d'anomalies non supervisé basé sur des arbres de décision aléatoires. Son principe repose sur l'observation suivante : les anomalies sont des points *rare*s et *différents*, donc plus facilement *isolables* par des coupures aléatoires de l'espace des features.

Pour chaque point x , l'algorithme construit un ensemble d'arbres binaires (forêt) en séparant récursivement les données par des hyperplans aléatoires. Le score d'anomalie est basé sur la profondeur moyenne à laquelle le point est isolé :

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \quad (6.1)$$

où $h(x)$ est la profondeur d'isolation de x , $c(n) = 2H(n-1) - \frac{2(n-1)}{n}$ est la profondeur moyenne d'un arbre BST sur n points, et H est la série harmonique. Un score proche de 1 indique une forte probabilité d'anomalie, un score proche de 0,5 indique un point normal.

6.2.2 Comparaison avec les alternatives

TABLE 6.1. Comparaison des algorithmes de détection d'anomalies

Algorithme	Supervisé	Faible dim.	Rapide	Interprétable	Retenu
Seuils fixes	Non	✓	✓	✓	Complémentaire
Isolation Forest	Non	✓	✓	✓	✓
Autoencoder	Non	✗	✗	✗	✗
One-Class SVM	Non	✓	✗	✗	✗
LOF	Non	✓	✗	✓	✗

6.3 Architecture du microservice Flask

Le microservice est développé en Python avec Flask. Il expose trois endpoints :

- `POST /train` — reçoit un fichier CSV multipart, entraîne le modèle ;
- `POST /predict` — reçoit `{ph, ntu}`, retourne le résultat d'analyse ;
- `GET /status` — retourne l'état et les métriques du modèle.

6.4 Pipeline d'entraînement

```

1 from sklearn.ensemble import IsolationForest
2 from sklearn.preprocessing import StandardScaler
3
4 # 1. Chargement et validation du CSV
5 df = pd.read_csv(io.StringIO(file.read().decode("utf-8")))
6 # Detection flexible des colonnes ph et ntu
7
8 # 2. Nettoyage
9 df = df[["ph", "ntu"]].dropna()
10 df["ph"] = pd.to_numeric(df["ph"], errors="coerce")
11 df["ntu"] = pd.to_numeric(df["ntu"], errors="coerce")
12 X = df.values
13
14 # 3. Normalisation
15 scaler = StandardScaler()
16 X_scaled = scaler.fit_transform(X)
17
18 # 4. Entraînement
19 model = IsolationForest(

```

```
20     n_estimators=200,
21     contamination=0.05, # 5% d'anomalies supposees
22     random_state=42
23 )
24 model.fit(X_scaled)
25
26 # 5. Sauvegarde
27 joblib.dump(model, "model_aqua.pkl")
28 joblib.dump(scaler, "scaler_aqua.pkl")
```

Listing 6.1. Pipeline d'entraînement – ai_service.py

6.5 Pipeline d'inférence

L'inférence applique une double logique : vérification des seuils métier puis score Isolation Forest.

```
1 @app.route("/predict", methods=["POST"])
2 def predict():
3     ph = float(request.json["ph"])
4     ntu = float(request.json["ntu"])
5
6     # 1. Seuils metier
7     raisons = []
8     if not (6.0 <= ph <= 9.0):
9         raisons.append(f"pH hors plage : {ph:.2f}")
10    if not (0.0 <= ntu <= 200.0):
11        raisons.append(f"Turbidite hors plage : {ntu:.1f} NTU")
12
13    # 2. Isolation Forest
14    X = scaler.transform([ph, ntu])
15    pred = model.predict(X)[0] # -1 = anomalie
16    score = model.score_samples(X)[0] # plus negatif = plus anormal
17
18    # 3. Normalisation du score [0, 1]
19    score_norm = float(np.clip((-score) / 0.7, 0.0, 1.0))
20
21    return jsonify({
22        "anomalie": bool((pred == -1) or len(raisons) > 0),
23        "score": score_norm,
24        "raison": " | ".join(raisons) or "Pattern inhabituel"
25    })
```

Listing 6.2. Pipeline d'inférence – ai_service.py

6.6 Persistance du modèle

Le modèle et le scaler sont sérialisés via `joblib` dans les fichiers `model_aqua.pkl` et `scaler_aqua.pkl`. Au démarrage du service, ces fichiers sont rechargés automatiquement s'ils existent, ce qui permet de conserver le modèle entraîné entre les redémarrages.

6.7 Workflow d'utilisation

1. L'ESP32 collecte les données pendant une période de fonctionnement *normal* du bassin (plusieurs heures ou jours);
2. Le fichier `data_aqua.csv` se constitue automatiquement sur le PC;
3. L'opérateur télécharge le CSV via le dashboard (*bouton « Télécharger CSV »*);
4. Il l'uploade dans la zone d'entraînement du dashboard (drag & drop ou sélection);
5. Le modèle est entraîné en quelques secondes et devient immédiatement opérationnel;
6. Chaque nouveau point de mesure reçu est automatiquement évalué.

Chapitre 7

Tableau de bord web

7.1 Architecture frontend

Le dashboard est développé en HTML/CSS/JavaScript vanilla, sans framework, afin de minimiser les dépendances et permettre un déploiement immédiat depuis le serveur Node.js. La seule dépendance externe est **Chart.js** (v4.4.1), chargée depuis un CDN.

7.2 Organisation en onglets

Le dashboard est organisé en deux onglets :

7.2.1 Onglet Dashboard

L'onglet principal présente :

- **Flux vidéo live** : image `` qui consomme le flux MJPEG du serveur ;
- **Panneau d'alertes** : liste dynamique des alertes actives (seuils dépassés + anomalie IA) avec code couleur vert/orange/rouge ;
- **Cartes métriques** : 5 cartes (température air, humidité, température eau, pH, turbidité) avec valeur courante, barre de progression et état d'alerte ;
- **Panneau IA** : score d'anomalie, statut, raison, stats du modèle et interface d'upload CSV pour l'entraînement.

7.2.2 Onglet Graphiques

L'onglet graphiques présente 5 courbes temps réel (Chart.js, type `line`) : pH, turbidité, température eau, température air et humidité. Un sélecteur permet de choisir la fenêtre d'affichage (30, 60, 120 ou 300 points). Les données sont stockées en mémoire dans des buffers côté client (`history`) et mises à jour toutes les 2 secondes.

7.3 Système de polling

Le dashboard interroge le serveur toutes les 2 secondes via deux endpoints :

```
1 setInterval(fetchData, 2000); // GET /data -> capteurs + graphiques
2 setInterval(fetchAI, 2000); // GET /ai -> score anomalie
```

Listing 7.1. Polling double – données + IA

7.4 Gestion des alertes

Le système d’alertes combine deux sources :

1. **Seuils côté client** : vérification JavaScript des valeurs reçues contre des seuils fixes (pH 6.5–8.5, NTU 0–1000, températures) ;
2. **Résultat IA** : le flag `anomalie` retourné par le microservice Python déclenche une alerte rouge avec la raison fournie.

7.5 Interface d’entraînement IA

La zone d’upload supporte le drag & drop natif (événements `dragenter`, `dragleave`, `drop`) et la sélection par dialogue. Le fichier est envoyé via `FormData` en `POST /train` et le résultat est affiché en temps réel (nombre de points d’entraînement, pH et NTU moyens).

Chapitre 8

Résultats et validation

8.1 Tests du firmware

8.1.1 Lecture des capteurs

Les capteurs ont été validés individuellement via la console série de l'ESP32 :

TABLE 8.1. Résultats de validation des capteurs

Capteur	Valeur test	Valeur mesurée	Écart	Remarque
DS18B20	18°C (bain contrôlé)	17,87°C	0,13°C	Dans la spec ($\pm 0,5^{\circ}\text{C}$)
DHT22	20°C / 60% HR	20,4°C / 60,6%	0,4°C / 0,6%	Normal
PH-4502C	pH 7,0 (eau distillée)	3,46	–	Calibration requise
Turbidité	Eau claire	541 NTU	–	Capteur hors de l'eau

Calibration du capteur pH

Les valeurs de pH mesurées (3.5) indiquent que le capteur n'a pas encore été calibré avec les solutions tampon appropriées. L'offset `PH_OFFSET` dans `config.h` doit être ajusté après calibration à pH 4.0 et pH 7.0.

8.2 Tests du modèle IA

Le modèle Isolation Forest a été testé avec un jeu de données synthétique de 20 points normaux (pH 7.0–7.6, NTU 12–25). Les résultats de prédiction sur des valeurs test montrent le comportement attendu :

TABLE 8.2. Résultats de prédiction du modèle IA sur données test

pH	NTU	Score IA	Anomalie	Raison
7.2	18	0.12	Non	Paramètres normaux
7.4	22	0.08	Non	Paramètres normaux
3.5	542	0.94	Oui	pH hors plage + Turbidité hors plage
9.8	15	0.71	Oui	pH hors plage
7.1	850	0.68	Oui	Turbidité hors plage
6.0	20	0.43	Oui	pH hors plage

8.3 Performance du système

TABLE 8.3. Métriques de performance du système complet

Métrique	Valeur	Commentaire
Fréquence d'acquisition	2 s	Paramétrable dans <code>config.h</code>
Latence ESP32 → dashboard	< 3 s	Envoi + appel IA + polling
Temps d'entraînement IA (20 pts)	< 0,5 s	Sur PC standard
Temps d'inférence IA	< 50 ms	Appel HTTP local
Débit vidéo MJPEG	10 fps	Dépend du réseau Wi-Fi
Consommation ESP32 (mesure)	240 mA	En transmission continue

Chapitre 9

Conclusion et perspectives

9.1 Bilan du projet

Le projet aqua-ai-monitor a permis de concevoir et d'implémenter une plateforme de surveillance aquacole complète, intégrant les technologies IoT, le développement firmware embarqué, l'ingénierie logicielle côté serveur et l'intelligence artificielle.

Les objectifs initiaux ont été partiellement atteints :

TABLE 9.1. Bilan des objectifs

Objectif	Statut	Commentaire
Acquisition multi-capteurs (5 variables)	✓	Fonctionnel
Transmission Wi-Fi HTTP/JSON	✓	Stable
Enregistrement CSV automatique	✓	Fonctionnel
Détection d'anomalies IA	✓	Modèle IF opérationnel
Dashboard temps réel avec graphiques	✓	5 courbes + alertes
Flux vidéo MJPEG	✓	ESP32-CAM → dashboard
Conception PCB	✓	Gerbers exportés
Calibration pH	✗	À finaliser en conditions réelles
Module IA embarqué (TFLite ESP32)	✗	Perspective future

9.2 Compétences développées

Ce projet a permis de mobiliser et développer des compétences dans plusieurs domaines :

- **Électronique embarquée** : programmation ESP32, protocoles OneWire/I²C/ADC, conception PCB sous KiCad, chaîne de mesure capteur → ADC → traitement numérique ;
- **Développement logiciel** : architecture client-serveur, protocoles HTTP, streaming MJPEG, programmation asynchrone Node.js, microservices Python/Flask ;

- **Intelligence artificielle** : machine learning non supervisé, Isolation Forest, normalisation des données, évaluation des performances d'un modèle de détection ;
- **Développement web** : HTML/CSS/JS vanilla, Chart.js, interface temps réel, drag & drop, gestion d'état côté client.

9.3 Perspectives d'amélioration

9.3.1 Court terme

- **Calibration des capteurs** : acquisition de solutions tampon pH certifiées et calibration 2-points du capteur PH-4502C ; étalonnage NTU avec solutions Formazine ;
- **Sécurité** : externalization des credentials Wi-Fi (NVS ESP32, SmartConfig) ;
- **Persistance IA** : amélioration du modèle avec accumulation progressive de données réelles du bassin cible.

9.3.2 Moyen terme

- **Capteur O₂ dissous** : ajout d'un capteur d'oxygène dissous (SEN0237-A ou similaire) pour une surveillance plus complète ;
- **Alertes push** : intégration d'une notification par email ou SMS (API Twilio, SendGrid) lors de la détection d'une anomalie ;
- **Dashboard responsive** : adaptation mobile du tableau de bord ;
- **Authentification** : ajout d'une authentification basique pour l'accès au dashboard.

9.3.3 Long terme

- **IA embarquée** : portage du modèle de détection sur l'ESP32 via TensorFlow Lite for Microcontrollers, permettant une détection locale sans dépendance réseau ;
- **Apprentissage en ligne** : mise en place d'un mécanisme d'adaptation continue du modèle à l'évolution des conditions normales du bassin ;
- **Analyse vidéo IA** : utilisation du flux OV2640 pour la détection comportementale des poissons (anomalies de mouvement, présence de pathologies visibles) par un modèle de vision artificielle ;
- **Multi-bassin** : extension de l'architecture pour surveiller plusieurs bassins simultanément avec un tableau de bord unifié.

Références bibliographiques

Bibliographie

- [1] F. T. Liu, K. M. Ting, Z.-H. Zhou, *Isolation Forest*, in *Proc. IEEE ICDM*, pp. 413–422, 2008.
- [2] Espressif Systems, *ESP32 Technical Reference Manual*, v5.2, 2023. [Online]. Available : https://www.espressif.com/documentation/esp32_technical_reference_manual_en.pdf
- [3] PlatformIO, *PlatformIO IDE for Embedded Development*, [Online]. Available : <https://platformio.org>
- [4] F. Pedregosa *et al.*, *Scikit-learn : Machine Learning in Python*, *JMLR*, vol. 12, pp. 2825–2830, 2011.
- [5] KiCad EDA, *KiCad 7 Reference Manual*, [Online]. Available : <https://docs.kicad.org>
- [6] FAO, *The State of World Fisheries and Aquaculture 2022*, Rome : FAO, 2022. ISBN 978-92-5-136364-5.
- [7] Maxim Integrated, *DS18B20 Programmable Resolution 1-Wire Digital Thermometer*, Datasheet Rev. 3, 2019.
- [8] Atlas Scientific, *pH Electrode and Circuit Design Notes*, Technical Note, 2021.
- [9] Chart.js contributors, *Chart.js v4 Documentation*, [Online]. Available : <https://www.chartjs.org/docs/>
- [10] OpenJS Foundation, *Express.js 4.x API Reference*, [Online]. Available : <https://expressjs.com>

Annexe A

Annexe A — Schéma de câblage ESP32

TABLE A.1. Câblage complet ESP32 – capteurs

Capteur	Pin capteur	Pin ESP32	Remarque
DS18B20	DATA	GPIO 4	Résistance pull-up 4.7 k Ω
DS18B20	VCC	3.3 V	
DS18B20	GND	GND	
DHT22	DATA	GPIO 15	Pull-up 10 k Ω recommandé
DHT22	VCC	3.3 V	
DHT22	GND	GND	
PH-4502C	PO	GPIO 34	ADC1, atténuation 11 dB
PH-4502C	VCC	5 V	Module nécessite 5 V
PH-4502C	GND	GND	
Turbidité	SIG	GPIO 35	ADC1, atténuation 11 dB
Turbidité	VCC	5 V	
Turbidité	GND	GND	
OLED SSD1306	SDA	GPIO 21	I ² C, pull-up 4.7 k Ω
OLED SSD1306	SCL	GPIO 22	I ² C
OLED SSD1306	VCC	3.3 V	
OLED SSD1306	GND	GND	

Annexe B

Annexe B — Guide de démarrage rapide

B.1 Prérequis

- Node.js ≥ 18 (<https://nodejs.org>)
- Python ≥ 3.11 (<https://python.org>)
- PlatformIO (<https://platformio.org>)

B.2 Installation

```
1 # Terminal 1 -- Service IA Python
2 cd esp32_cam_stream
3 python -m pip install flask flask-cors scikit-learn pandas numpy
   joblib
4 python ai_service.py
5
6 # Terminal 2 -- Serveur Node.js
7 cd esp32_cam_stream
8 npm install multer form-data node-fetch@2
9 node server.js
```

Listing B.1. Installation et démarrage du serveur

B.3 Flash du firmware ESP32

```
1 cd esp32_code/esp32_aqua
2 # Editer src/config.h avec le SSID, mot de passe et IP du serveur
3 pio run --target upload
4 pio device monitor --baud 115200
```

Listing B.2. Flash via PlatformIO CLI

B.4 Premier entraînement du modèle

1. Laisser tourner le système 30 minutes en conditions normales ;
2. Ouvrir <http://localhost:3000> ;
3. Cliquer « *Télécharger CSV* » pour récupérer `data_aqua.csv` ;
4. Dans l'onglet Dashboard → section IA → glisser le CSV dans la zone d'upload ;
5. Cliquer « *Entraîner* » — le modèle est opérationnel en moins d'une seconde.

Annexe C

Annexe C — Structure du dépôt GitHub

```
1 aqua-ai-monitor/  
2   README.md  
3   Fiche_sujet_SPI_2.pdf  
4   esp32_code/  
5     esp32_aqua/                -- Firmware capteurs (PlatformIO)  
6       src/  
7         main.cpp  
8         config.h  
9         dht_sensor.cpp/h  
10        ds18b20_sensor.cpp/h  
11        ph_sensor.cpp/h  
12        turbidity_sensor.cpp/h  
13        screen_oled.cpp/h  
14        http_sender.cpp/h  
15        platformio.ini  
16    esp32_cam/                  -- Firmware camera (PlatformIO)  
17        src/  
18          main.cpp  
19          camera.cpp/h  
20          streamer.cpp/h  
21          config.h  
22    esp32_cam_stream/          -- Serveur + dashboard + IA  
23        server.js  
24        ai_service.py  
25        requirements.txt  
26        public/  
27          index.html  
28    SEBB_aqua/  
29        carte_aqua/            -- Projet KiCad PCB  
30          carte_aqua.kicad_sch  
31          carte_aqua.kicad_pcb  
32          gerbers/
```

Listing C.1. Arborescence du dépôt aqua-ai-monitor